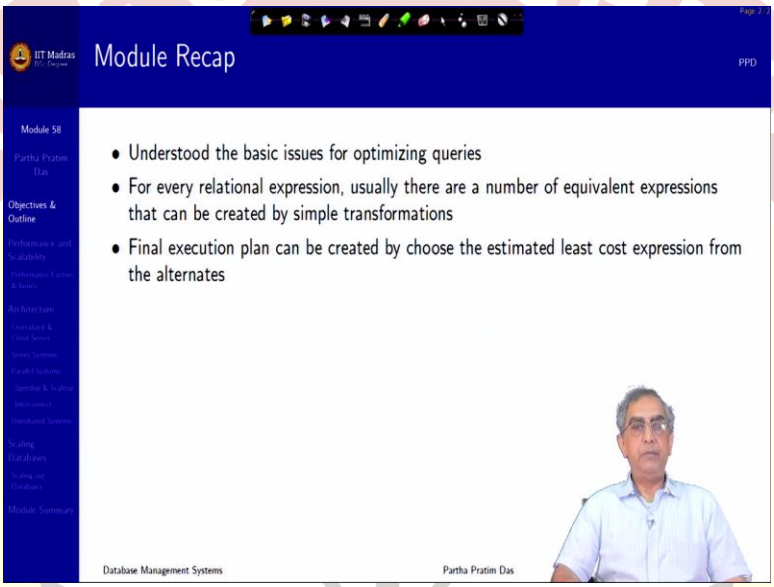
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
RDBMS Performance & Architecture

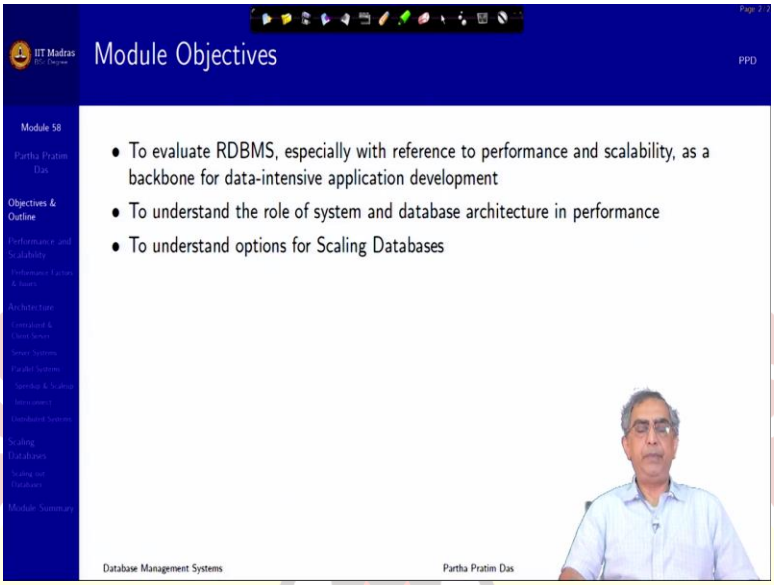
Welcome to Module 58, of Database Management Systems course in the IIT Madras online BSc program.

(Refer Slide Time: 00:29)



In the last module, we talked about the basic issues of optimizing queries. Every relational expression has a set of number of equivalent expressions, then can be created through equivalence rule transformations. And the final execution rule can be created by choosing the least estimated cost expression from the alternates, and that gives us a good optimized query.

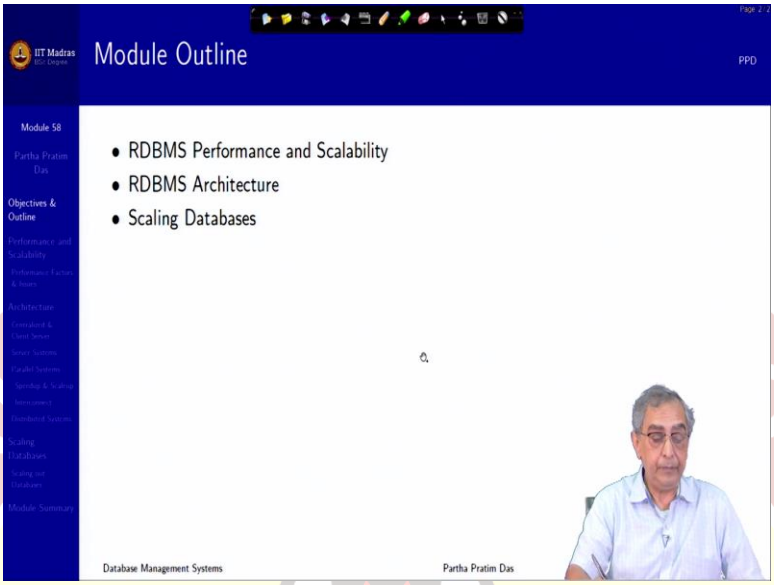
(Refer Slide Time: 01:00)



In this module, we will now kind of do a evaluation of RDBMS. We have learned about all the major aspects of an RDBMS, starting from the basic capture of modeling to design to normalization to query formulation to writing of transactions, concurrency, backup, optimization, all of that.

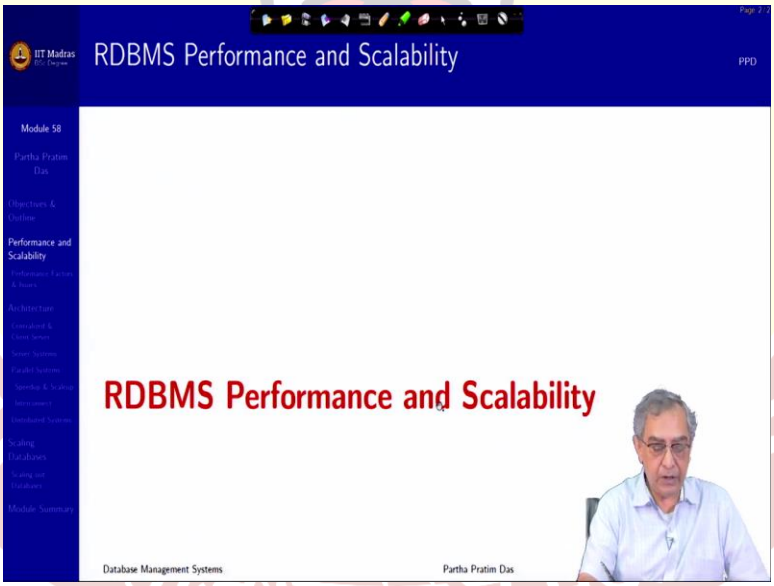
Now, overall, let us see with reference to performance and scalability, how does RDBMS perform as a backup or backbone rather, backbone for data intensive development, data intensive applications. So, we will see specifically the role of system and database architecture on performance. And we will understand the options for scaling databases to larger and larger size.

(Refer Slide Time: 02:02)



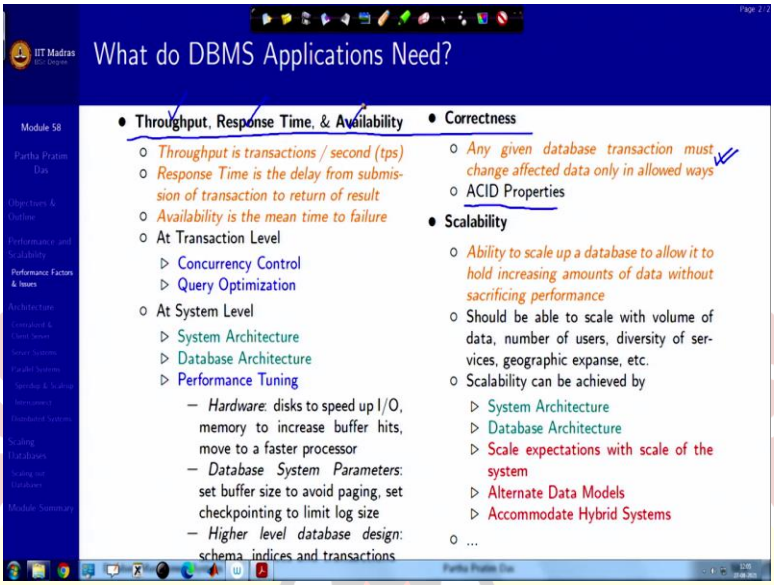
And these are the three specific items, we will talk about.

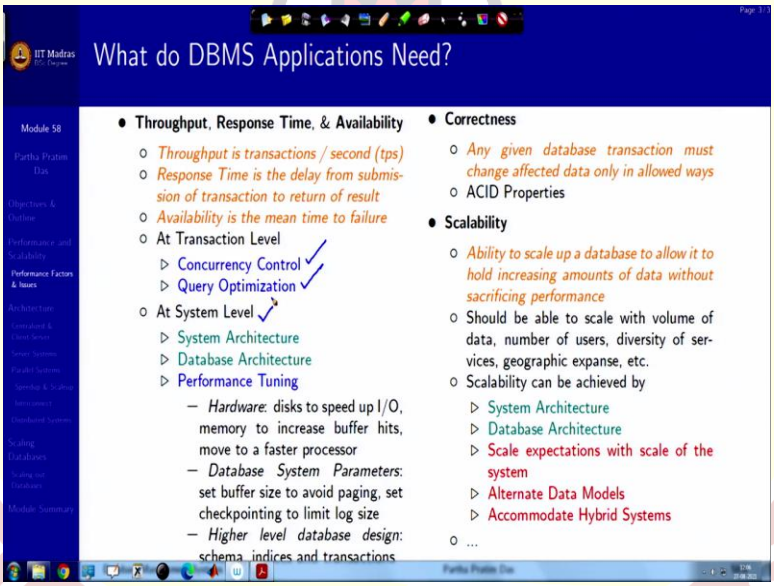
(Refer Slide Time: 02:07)

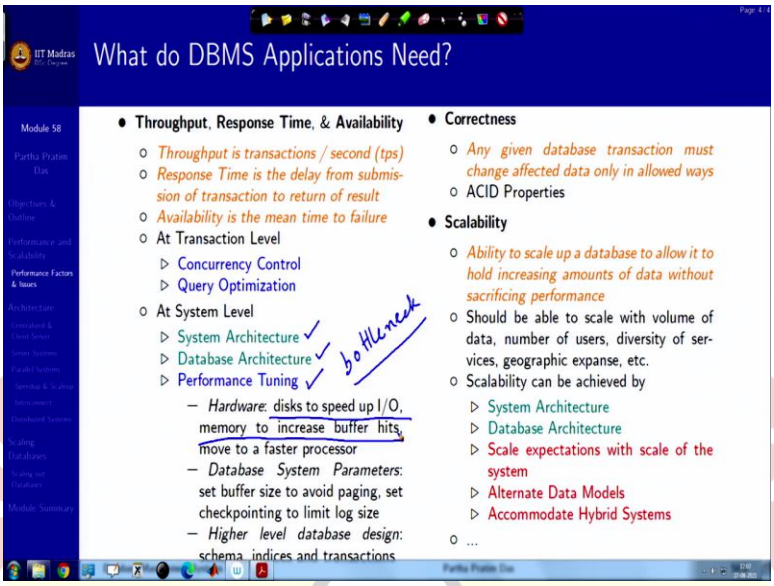


So, first let us talk about the performance and scalability.

(Refer Slide Time: 02:10)







So, this is what you have, we have learned through the course. That, what I need in a database application is obviously primary is correctness. Any given database transaction must change the affected data only in the allowed ways. And we have seen, that this is guaranteed by ensuring the ACID properties. In terms of performance, there are three primary factors, that are important. One is throughput, that is how many transactions per second can be performed, called the TPS of the system.

Second is response time, response time is the delay from the submission of a transaction to the return of the result. So, the response time also has to be. So, TPS should be high and response time should be low, as low as possible. And then the availability is decided by the mean time to failure, which we also expect to be high and we have talked about several, options for ensuring all these.

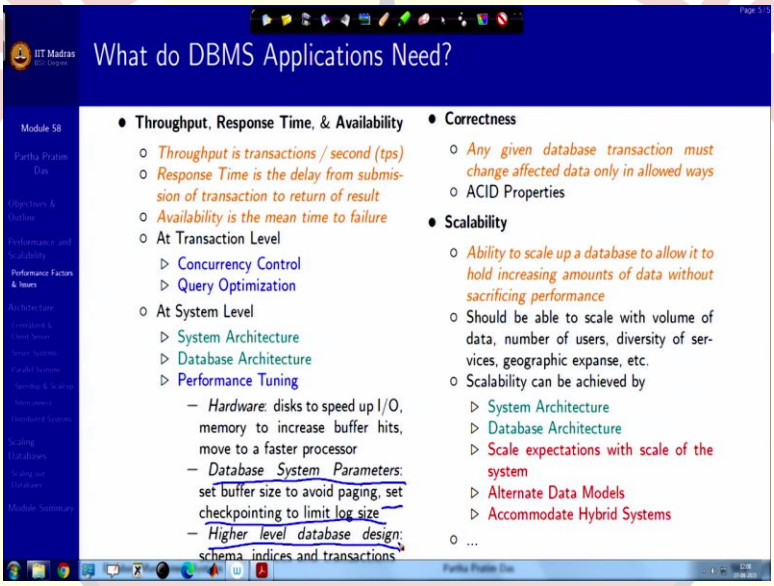
At a transaction level, these are primarily guaranteed by concurrency control, query optimization and few of the other things like use of raid and so, on. At a system level, that is if I do not look at specific transactions or specific groups of transaction, if I want to look at the entire database system as a whole then these are primarily guided by the system architecture and the database architecture.

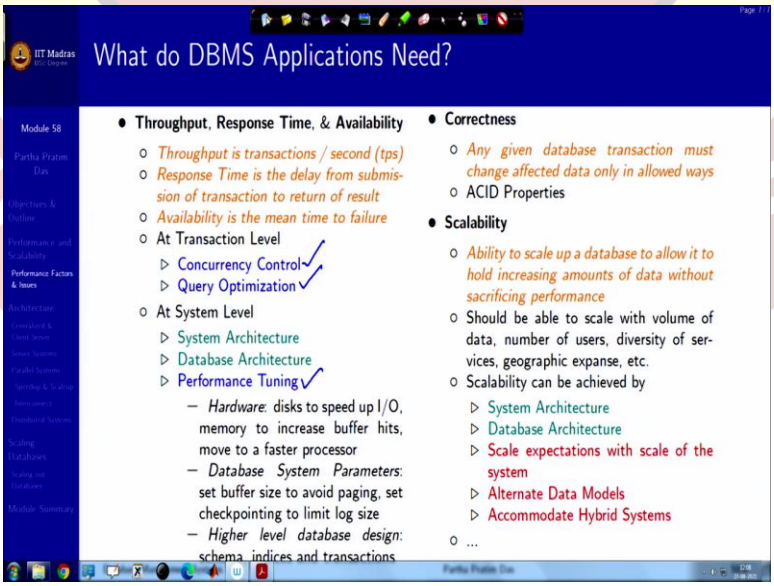
System architecture talks about the machines, the memory, the disk, the connectivity and so on. And database architecture primarily talks about how you run different database processes over this system architecture. Certainly along with this comes the issues of performance tuning, that is

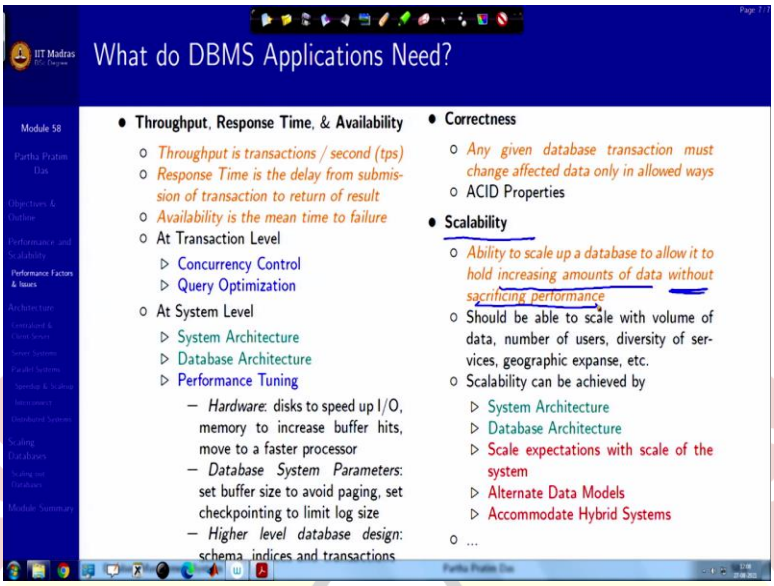
if you find that the database is not having the required performance in terms of these parameters, then you can decide, analyze and see where is the bottleneck.

And that bottleneck, so, performance tuning is primarily related to analysis of bottleneck and removal of bottleneck. So, it may be in terms of hardware tuning, you may have discs to more days to speed up the IO. You can add more memory to increase the buffer hits, the more number of blocks can reside together, you can move to a faster processor and so on.

(Refer Slide Time: 04:52)



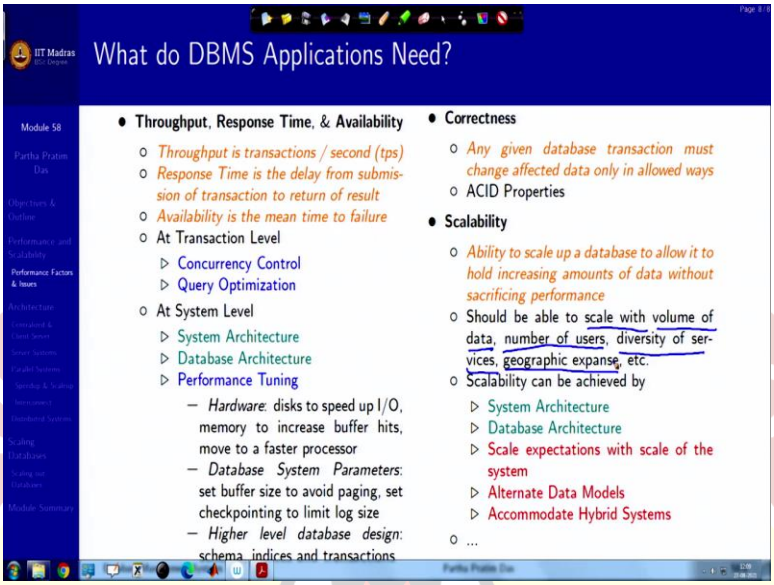


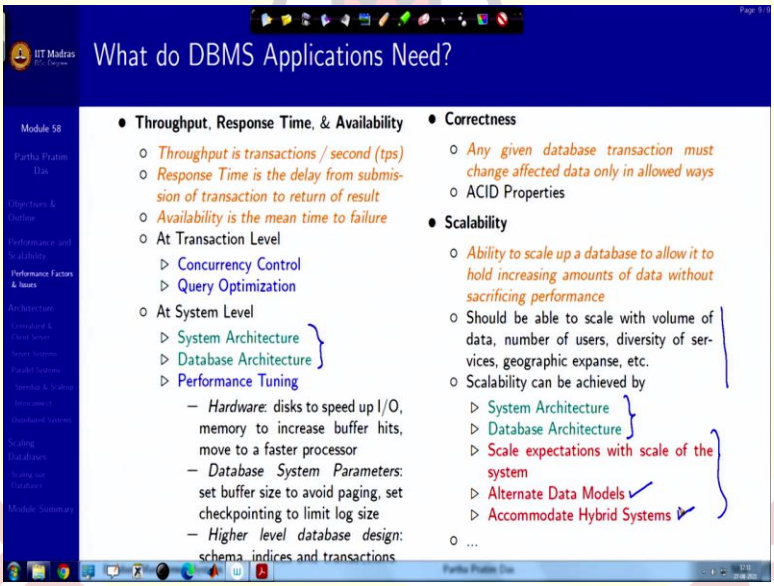


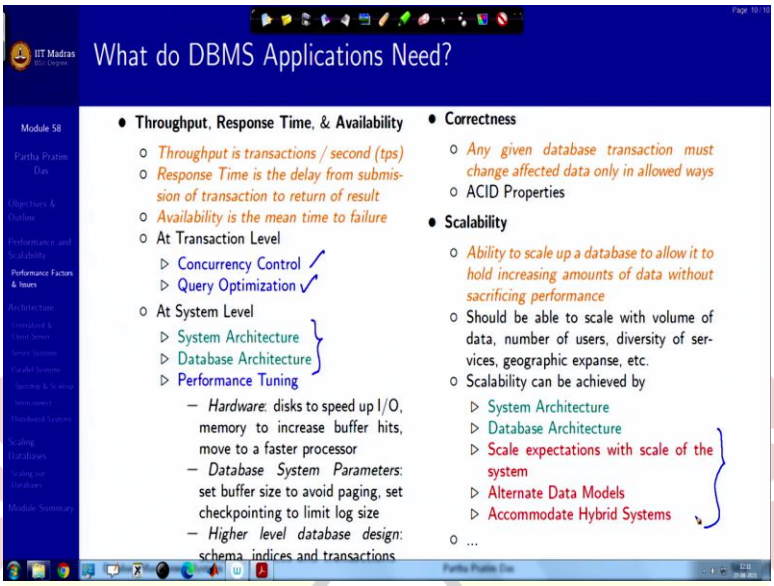
In terms of the database system parameters, you can tune a number of parameters like the size, set the size of the buffer, so that you can avoid frequent paging. Paging naturally means that a lot of times you are writing back and accessing the same page. You can set proper check pointing to limit the log size. Whenever a failure happens, we have to go back to the checkpoint and then redo and undo using the log.

So, from the checkpoint the entire log has to be maintained in the hot backup. So, we can limit that tune, then. And we can have high level database design in terms of schema modification, index modification, transaction modifications and so on. So, this is what we have already seen, anything that you see in blue here is what we have in depth or in, at a surface we have discussed. Now comes the question of scalability, how big the database or the application can get? The ability to scale up a database to allow it to hold increasing amounts of data, without sacrificing performance, increasing amounts of data is important. And without sacrificing performance is important. If you have more and more data, and if the throughput falls off, or if the response time gets bad, then the system is not scalable.

(Refer Slide Time: 06:28)







So, in generality, a database should be able to scale with volume of data, that I can increase the data itself. There could be an very sharp increase in the number of users, there could be increase in terms of the diversity of services that I provide. So, the transactions and applications on those transactions. Or it could be based on the geographic expanse of the database, I mean, just in your organization, in your office or in your city or in your country, or across continents and so on.

So, these are the different factors, which is decided based on the type of application that you are going to develop. So, the question is, does RDBMS scale up properly? And what are the factors that decide the scale up? Again, the scale up can be scalability, can be primarily achieved by tuning the system architecture and the database architecture. This is, this is same as what you have in terms of the usual RDBMS application development.

But there could be issues which go beyond that, you want to scale to a point where even by tuning system architecture or database architecture, you may not be able to handle everything. So can you scale expectations with the scale of the system? So, now we are trying to say something a little different that well, we will expect something different from the database. For example, we have always insistent on 100 percent correctness.

Can we live with a little less consistency in the database, we will, we will get shocked to know that but well, those are those are alternatives that people have looked at. And those are effective very much. And we will have to define what is that reduced come consistency mean? We can

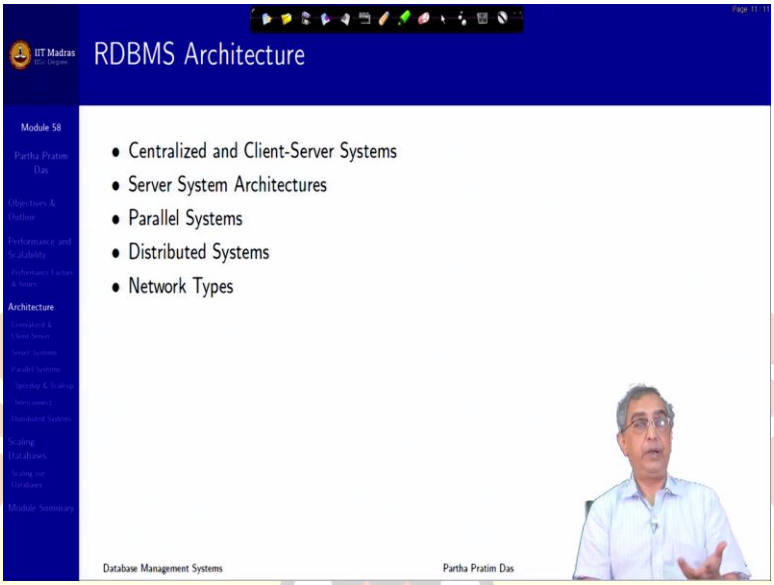
look at alternate data models, we can accommodate hybrid systems, a mixture of different types of systems and so on.

So, what I mean, in terms of this color codes here, these blue ones are what we have discussed. This is what I am going to discuss at a surface level in the current module. And these are the ones which will subsequently take up showing that, what is the route forward beyond a relational database system.

(Refer Slide Time: 08:44)

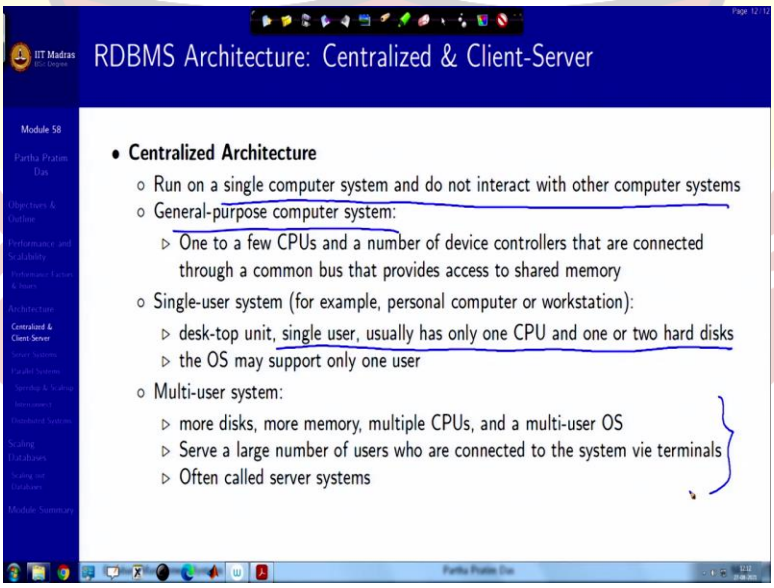
So, in the remaining of this module, we will focus on RDBMS architecture.

(Refer Slide Time: 08:51)



Now, RDBMS can be architected on a, on different ways. Centralized or client server systems, server systems can have multiple different architectures. Then we can have parallel systems, we can have distributed systems connected over different types of networks, and so on. So, let us take a quick look at these options.

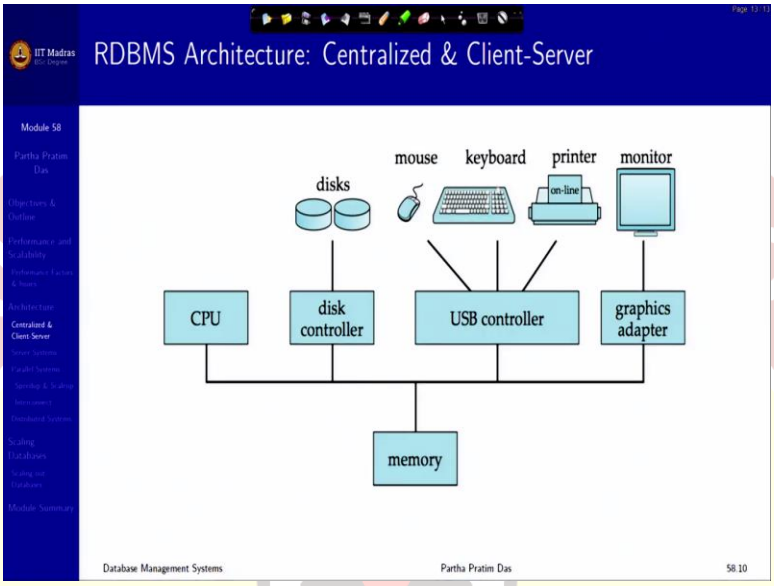
(Refer Slide Time: 09:12)



So, a centralized system is one which run on a single computer. Single computer system do not interact with other computer systems, it is usually a general purpose computer. So, for small application, single user application or even multi user small applications, these are really good

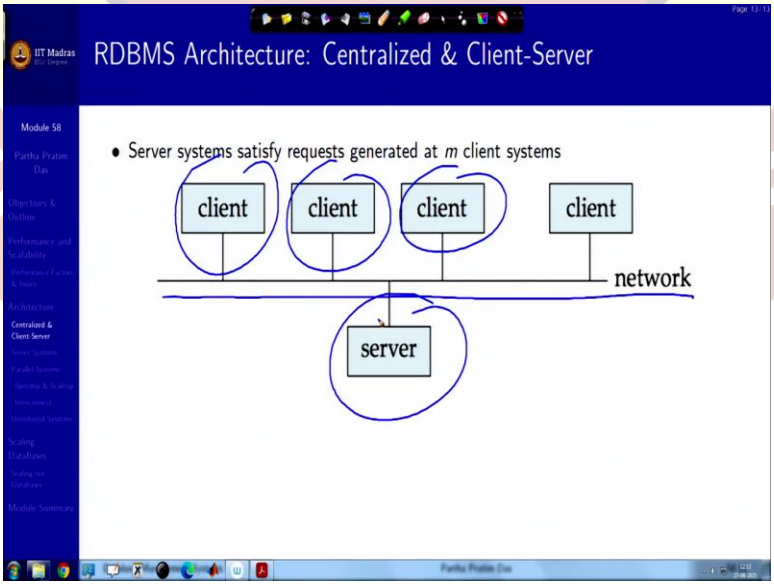
options. So, desktop unit, single user, usually only CPU one or two hard disks, and or you may have a multiple of them and have a little bit bigger, which we call, often call the server system.

(Refer Slide Time: 09:48)



Now centralized architecture is useful for very small applications. And this is typically how it looks like so you have everything connected together with your CPU with disk controller, USB controller, your graphics adapter, memory everything together.

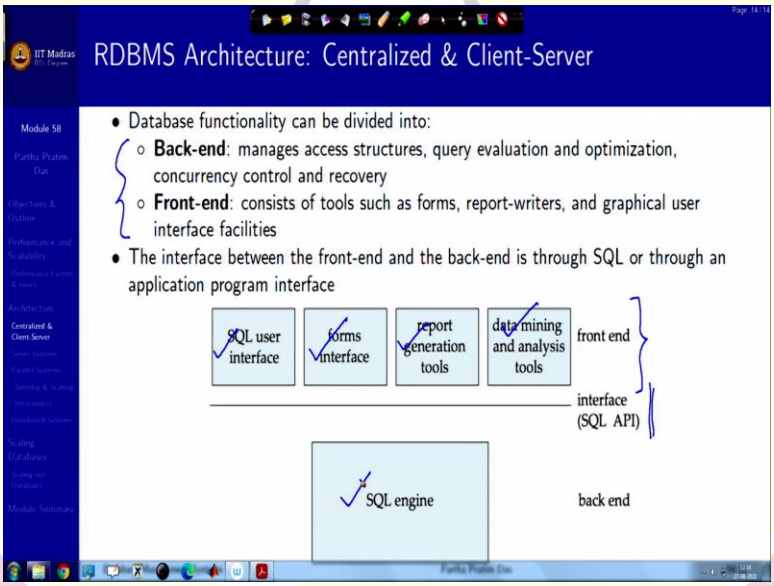
(Refer Slide Time: 10:01)



Now, what is more common is a client server architecture, where what you do is kind of you have a network, which a network which kind of separates the client and the server. Client is, who is asking for a request and the server is with providing that request. So, you may have different user clients running on one side of the network. And one or more servers running on the other side of the network, which provide the services, which makes things a little bit more scalable.

Actually, this is the first step in scalability that you can make, you can have more servers, you can have more clients and scale the system compared to the centralized system. And work in a better manner and all that we have discussed so far, are primarily based on the client server system.

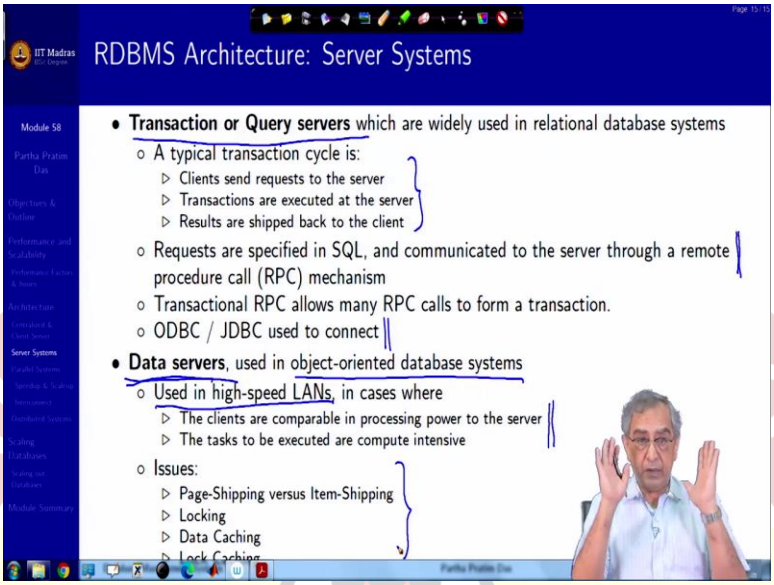
(Refer Slide Time: 10:52)



So, in this kind of a scenario, the functionality is typically divided between back end and front end, which we have discussed. We have discussed about two tier and three tier architecture for that, even n tier architecture for that. So, you have a front end, which runs the front-end tasks like SQL user interface, when you write the queries, you have forms, different kinds of forms. Report generation tools, data analysis, statistics, generation tools, and all that.

So, these are the different parts of the application that run on the front-end side, the client side. Then you have an interface, which is typically over a, over SQL API. So, you have ODBC, JDBC, these kinds of things we have discussed. And in the back end, you have the SQL engine, which could have, have one server or multiple servers.

(Refer Slide Time: 11:49)



So, if we look at this kind of a client server, then on the front-end side, you can see there are different applications and they could run on the same machine, client machine or multiple different machines. On the back end, on the server site, there again could be multiple different servers. And particularly, functionally, we look at two types of servers in this process. One is what is called the transactional query server.

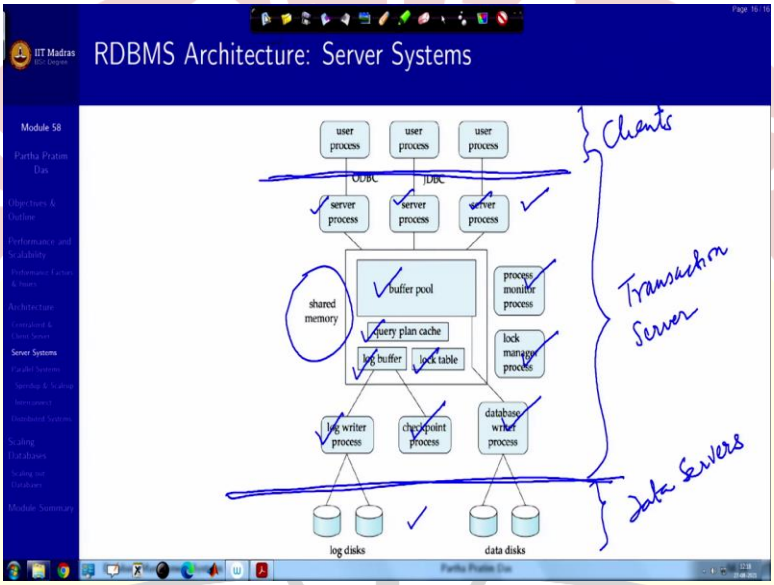
Which is widely used in a relational database system. You know a typical transaction lifecycle, that is clients send request to the server, I mean through those ODBC, JDBC, etc. Transactions are executed at the server. And the results are shipped back to the client again through that API's, again through those connectivity. So, the requests in are specified in SQL, communicated through a remote procedure call mechanism.

So as if remotely, you are calling a procedure, you have seen these mechanisms in terms of embedded SQL as well as connectivity oriented once. Some, RPC transactional RPC allowed many RPC calls to form a transaction and ODBC JDBC are used to connect. This is, this is the transactional part of the server. The other set of servers could actually (())(13:09) to the data. And it is particularly very important when you start, migrating or kind of making your relational database more object oriented.

So, they are used typically in high-speed land, because with the data service, the data exchange has to happen in a very fast rate. Otherwise, the whole system will performance will go down.

So, the clients are comparable in processing power to the server, the task to be executed is compute intensive. There are several issues like paid shipping, item shipping, locking, caching, of the data, caching of the locks and all that, that is a very detailed architectural and organization issues, we will not go into those. But just understand that these kinds of issues exist.

(Refer Slide Time: 14:01)



So, given that a typical server system architecture would look something like this. So, what you have here is the, is the interface layer. So, you have user processes on one side the clients and they are using ODBC JDBC to trigger corresponding server processes. And then as you can see, the this is basically your transaction servers. On this side, you have data servers, here you have your clients.

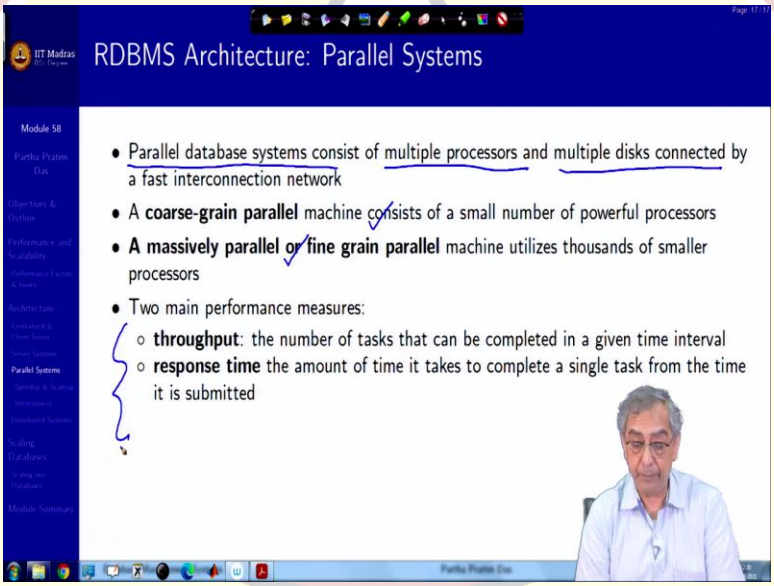
So, that is how, so this is this is a interface which is loose connected, this is an interface which may be a high throughput LAN or WAN system. Now in the transaction server you have process monitoring, that is what all is going on. Lock managers we have talked about, we have talked about checkpointing, log writing, database writing. So, all these different processes will run concurrently on the transaction servers, to manage the server processes that have been initiated by the client.

And what sits between them, typically in a shared memory or some other configuration are a number of buffers and queues that are maintained. So, you have a buffer pool, we learnt about that, we have log buffer occurring here. We have the log table occurring here, we have the query

plan caching occurring here and so on. So, this is the whole set of things that will happen in terms of the transaction server.

So, they could happen in a single server, they could happen in multiple server also, there could be microarchitectures that go into this. On the database server site, you can have the data on, on single disks or you may have multiple servers, which manage data between multiple disks, not only in terms of having redundancy, but also in terms of partitioning the data. We will look into those aspects. So, that is the overall, client server architecture that would happen for a typical relational database system.

(Refer Slide Time: 16:14)



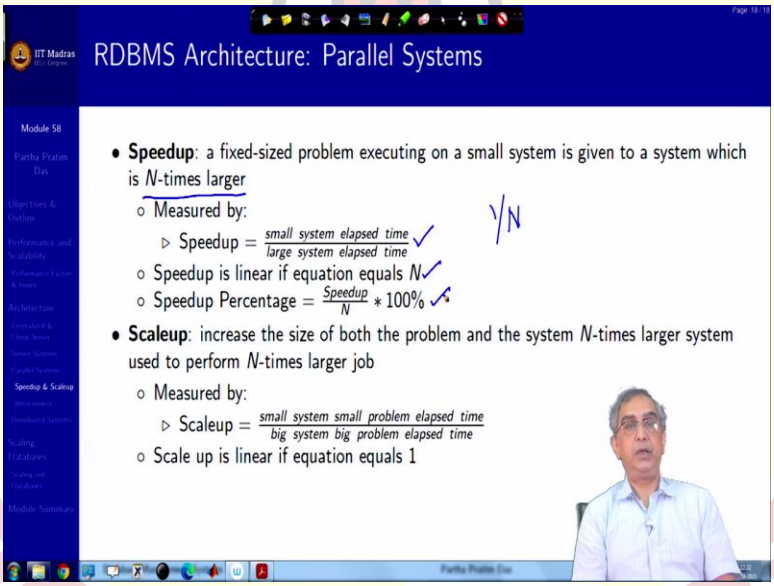
Now, going to the next level, what we can have are parallel database systems. Trying to get more throughput, what we can have parallel database systems. Where multiple processors and multiple disks are connected by a fast interconnection network. So, these are typically tightly coupled architecture. So, they can, they can do very fast transaction, I mean very fast data exchange between them.

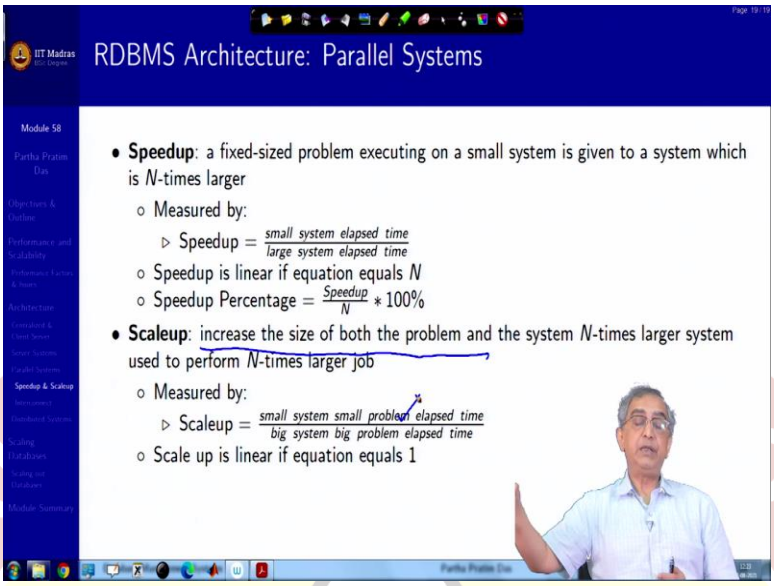
It could be coarse grained, that the machine consists of a small number of powerful processes. It could have a different number of parallel processes within them, itself or they may connect a number of processors together. And the other could be massively parallel or fine-grained parallel machine, that use thousands of smaller processes. So, various such parallel processing architecture is involved here.

And as you do things in parallel, it is not only the database considerations, concurrency and all that you will have to take care of, but you will also have to take care of the basic issues of parallel processing. The concurrency issues, because of parallel processing. Because you are now running multiple transactions not only concurrently, but you are running them parallely concurrently.

So, there are parallel a number of things are, are executing number of processes are executing on, on number of cores of a machine could be which are, which are in parallel. And on each one of them there is concurrency, and then there will have to be a parallel communication between them conflict resolution and so on. Naturally, the main performance measures are throughput and response time.

(Refer Slide Time: 18:15)





So, parallel architectures can be parallel systems can be used to, both for speed up as well as for scale up. Speed up is suppose, I have a fixed size problem, and it runs on a small system. Now, let me throw that problem to a larger system, which is n times larger. I am not very strictly defining what is n times larger, mean it, may mean n number of CPUs, it may mean n times memory lot of, but let just, let us say take it that it is, it is in a way qualitatively have n times more resources.

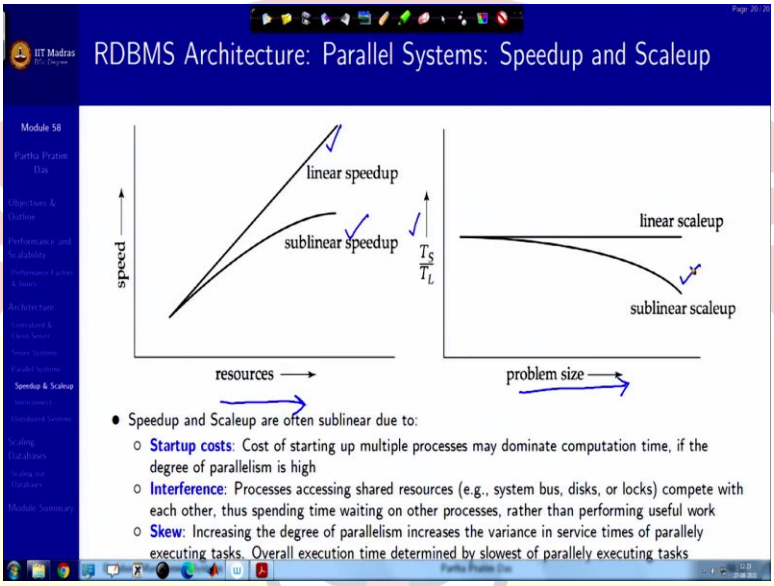
Now, the speed up is for that fixed size problem, the time that you take to execute it in the small system and the time that you take to execute it is in this, n times larger system. If you take their ratio, then you call it the speed up. Naturally, since the the system says, system is n times larger, so, the maximum speed up can be, that can be achieved is equal to n. You cannot so, you cannot have more speed up than n.

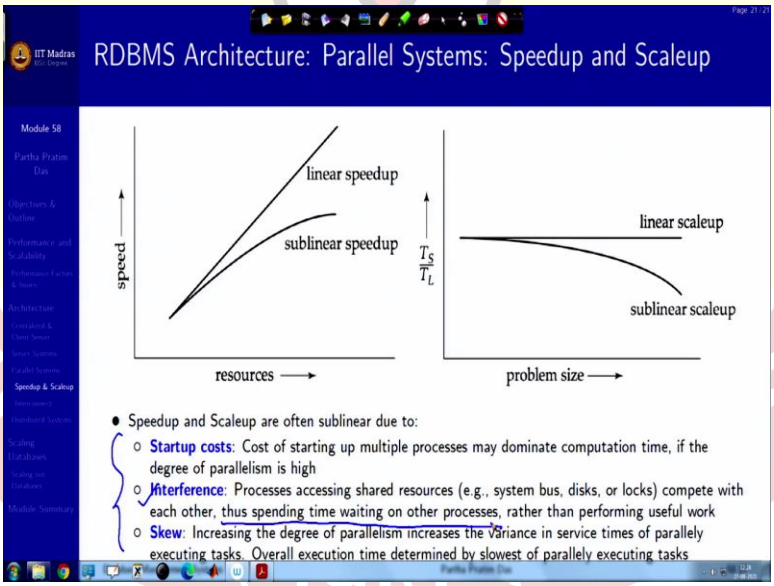
That is your execution time cannot go below one nth of what how it runs in the smaller system. Now, typically what we do? We normalize this and talk about speed up as a as a percentage or a fraction between 0 to 1. So, speed up by n times 100 percent is a speed up. Similarly, we have a concept of scaleup. if we increase the size of both the problem and the system n times larger to perform an n times larger job.

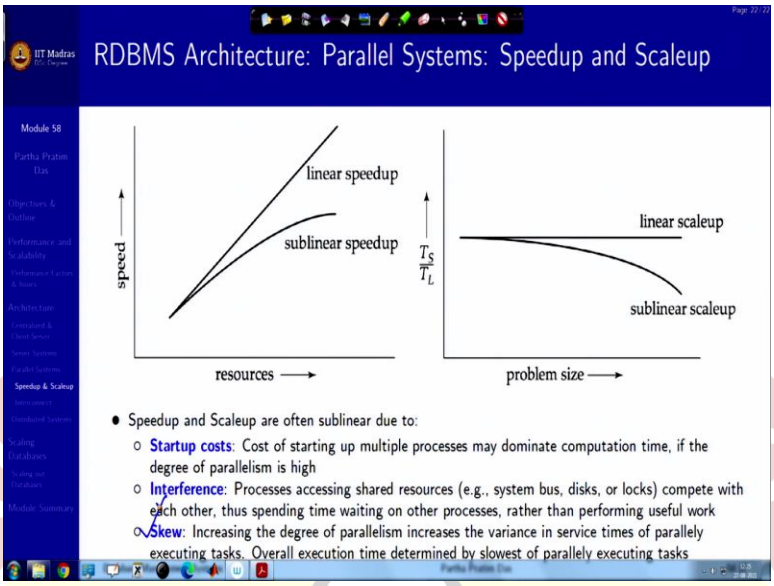
Earlier the job size was fixed, and we just increased the system. But if I, if I increase the job size also then the small system running on the small problem the time it takes and the big system running on the big problem the time it takes, what is that ratio? Naturally, it cannot be more than

1. You can even easily understand, because I can individually run the larger problem, larger n problems, n times larger problem rather in the n times larger system. So, it speed the scale up cannot be more than 1, but will, is it linear? Is it, is it 1? Or do we typically get less than that.

(Refer Slide Time: 20:35)







So, if you look at the typical plot of this, in terms of how it grows, you will see that as you grow the number of resources that you did in the, in the first case for speed up, the speed up expected is linear that the best. But it typically is sub linear, it does not, it is grows more slowly than linear and try to start flattening out. Similarly, if you take the second case, if you increase the problem size, then the scale up also does not remain flat linear, you would expect it to be linear.

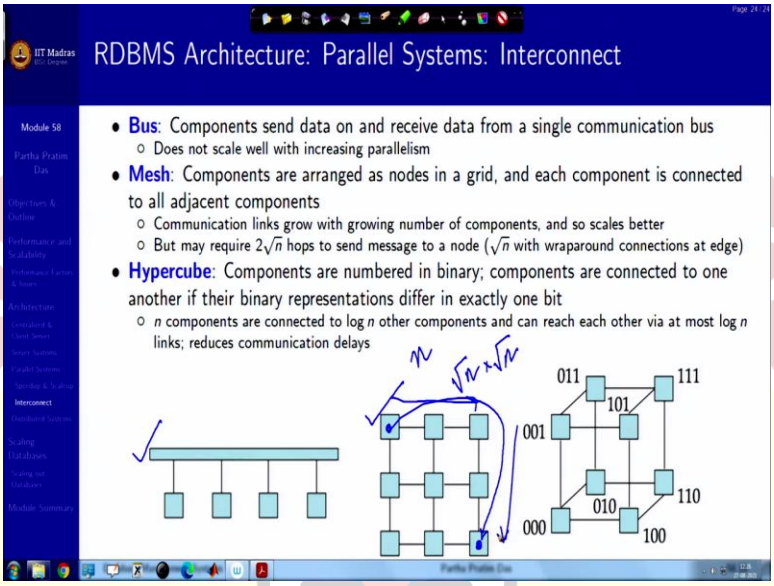
But it does not, rather it becomes sub linear, it starts to fall off. So, this will be very, very typical that you will observe and there are several reasons for which, for this one could be the I mean, since you are running multiple processes now in either case. So, the startup cost of starting multiple processes, it will start dominating if the degree of parallelism starts getting high. There could be interference.

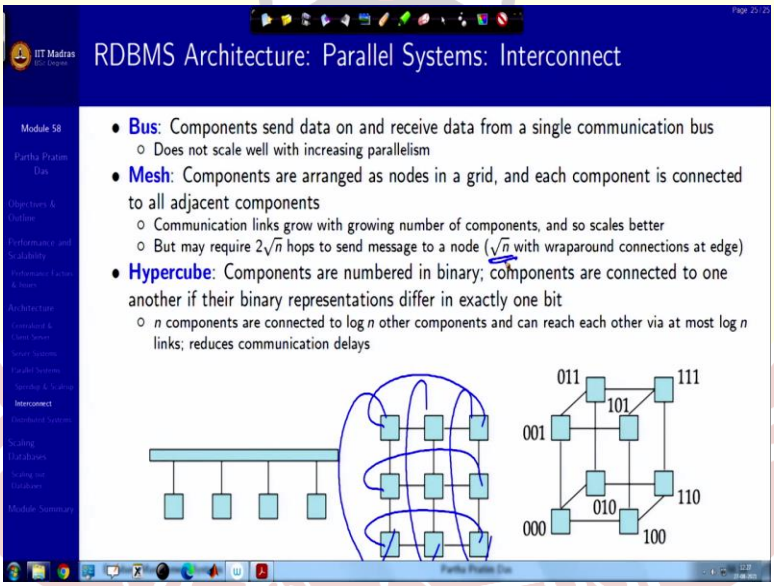
Because multiple shared processes, processes will share the system bus, the disk, the locks, so there will be a lot of conflict, blocking that will have to happen. And the lot of time will be maybe wasted by waiting on other processes then actually performing useful fruitful tasks. Also, the another reason that you get sub linear speed up or sub linear scale up is the fact that there is a, in real life there is a lot of skill.

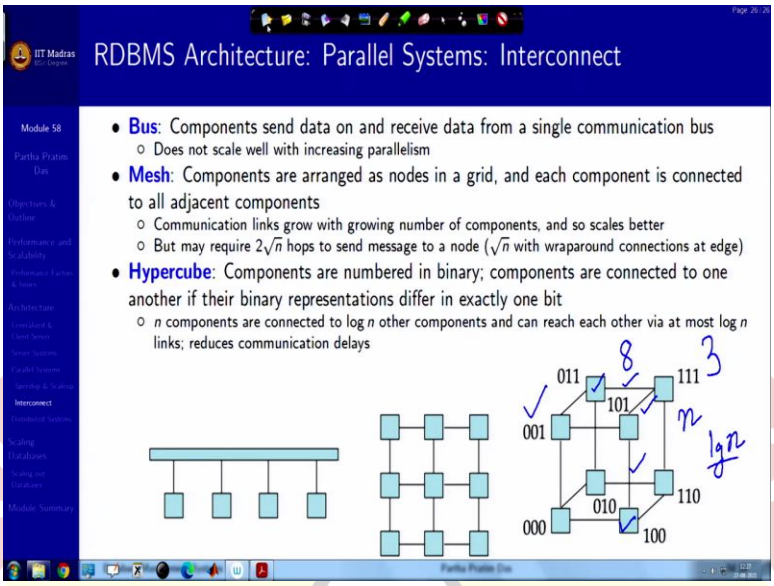
Then as we increase the degree of parallelism so, the increase in the variance of service time of parallel executing tasks, there are ten tasks. Their service time will be very, usually be very skew. That one will run very fast, one will take a lot of time and so on. So, the overall throughput or the overall speed up will depend on the execution time of the slowest parallely executing task.

So, the more the skew, you will have a more of a sub linear performance. So, these are some of the factors that will play against scale.

(Refer Slide Time: 22:52)







I mean continuing to scale up at a steady rate naturally, as I said the parallel systems will have to communicate between them in a very fast manner. So, it comes a question as to how to do you interconnect these parallel systems. Of course, this is not a core database topic, but this is something which need to be understood by the database engineer whose system runs on a parallel system.

So, you could connect them through a bus directly. So, if you connect them to a bus, this will not scale well with increasing parallelism, because any pair of processes you have to exchange data, we will have to hold that bus. So, a lot of things will get linearized. You could have a mesh, where you connect them in terms of kind of a square connection. So, if you have n processors, so this is a root n by root n mess mesh.

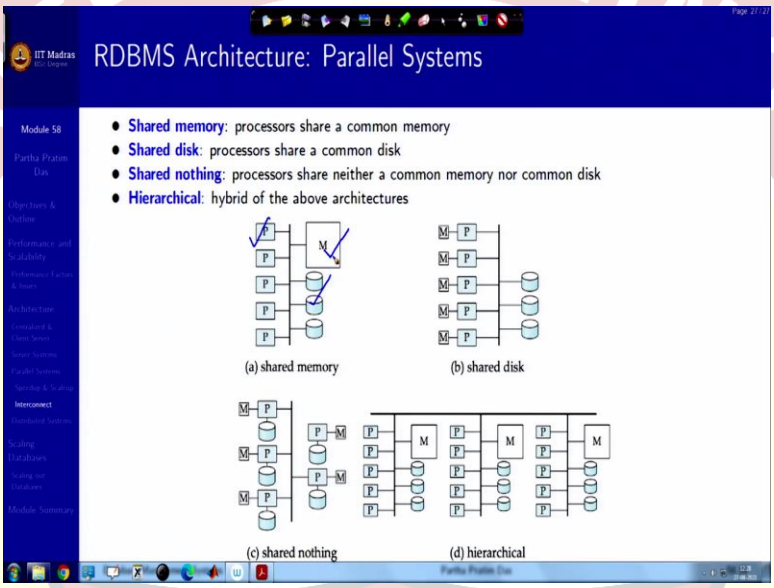
So, the longest time of communication that you will need is going from this end to the other. So, it is root n on this side and root n on this side. So, it could be twice of root n. If you use wraparound connection, that is instead of just having a mesh if you have a toroid, that is you have connections like this as well, then you will have a, or and like this, then naturally you will have a better connectivity and, and you will have a better data transfer rate.

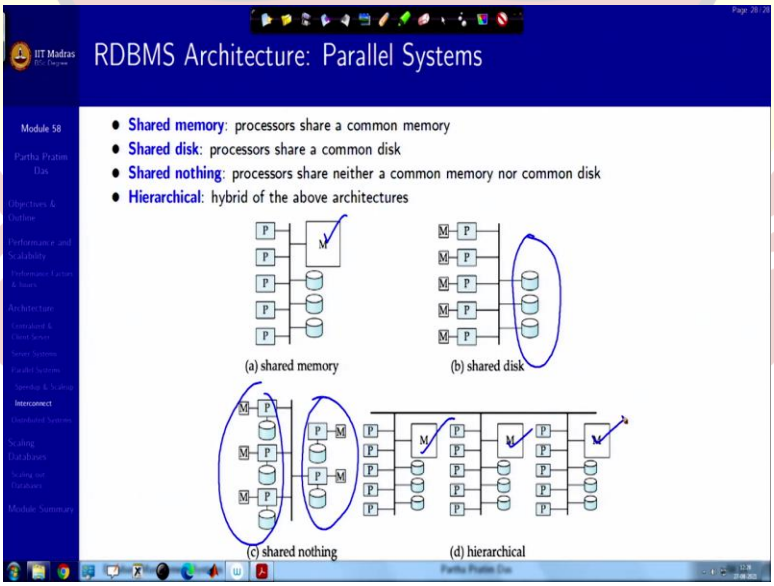
You will bring it down to half of that, you will bring it down to because it is basically the diameter of the graph, connectivity graph. Or you could use a hypercube, which is like this. So, here it has eight processors. And therefore, its maximum route between these two is one, two

three connections, three links. So, if you have n processes connected this way, then we will have a maximum transfer time which is log n.

So, which is obviously, so you can you can see this is kind of a hierarchy of improvements of connections. Obviously, as you go from bus to mesh to hypercube your interconnect costs are increasing and that way you will have to do a balancing of sorts.

(Refer Slide Time: 25:11)

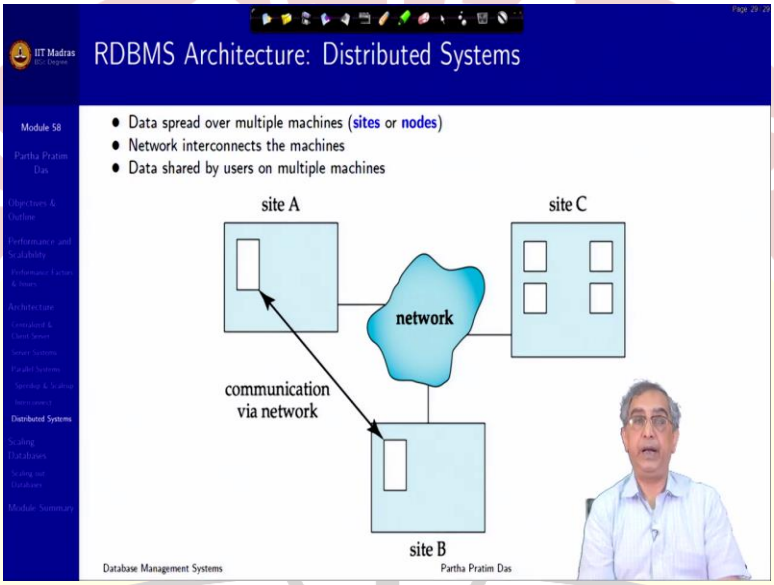




Now, there are different kinds of parallel systems have been designed, are available. You could have them in shared memory, there is one memory that everybody uses. These are processes and

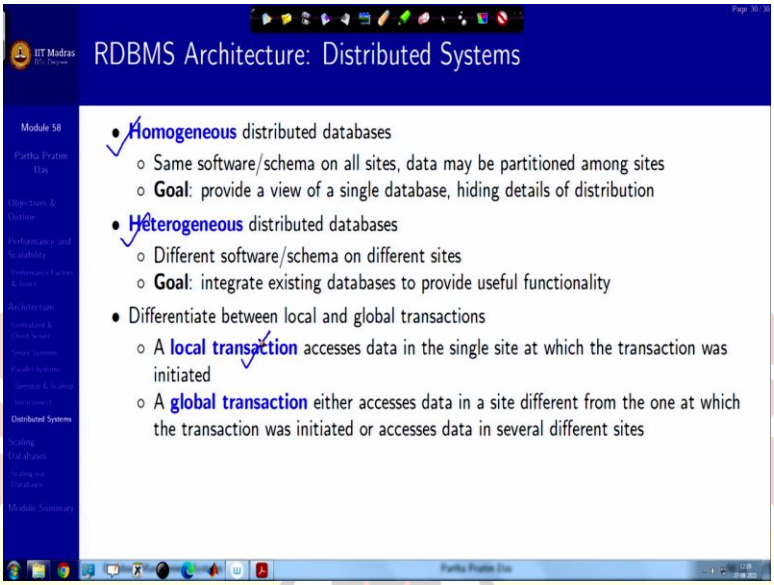
these are disks you can make out. So, it could be a shared memory, it could be shared disk through a bus. It could be shared nothing that everybody is independent by itself and just communicates or it could be a hybrid system that partly shared and then partly independent. So, these are the different parallel architectures that can happen.

(Refer Slide Time: 25:41)



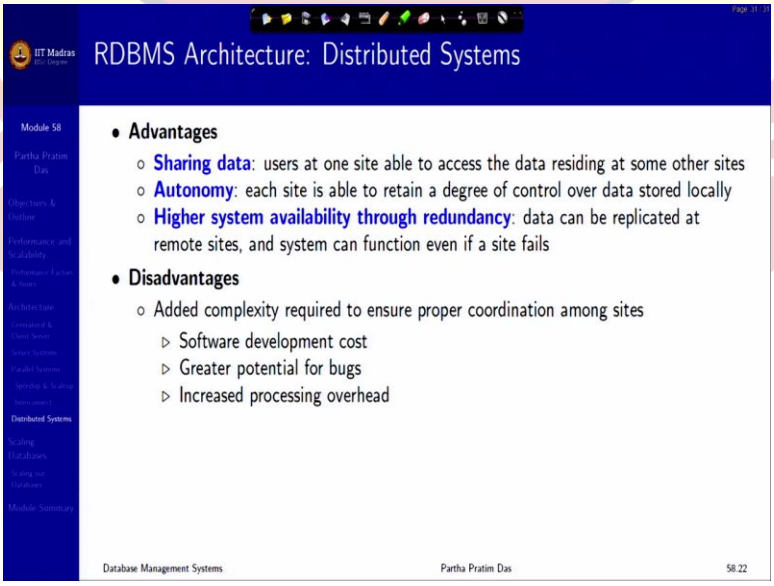
Going beyond, so this was about a tightly coupled system. But when the database gets even larger, what you would want is data is spread over multiple machines and or sites and nodes, which mean that all of the data may not be available at a particular node. It may be somewhere else, that part of the data is somewhere else. And network and relatively, a less bandwidth network will connect them. So, data is shared by users on multiple machines and yet free sites are what is so shown.

(Refer Slide Time: 26:19)



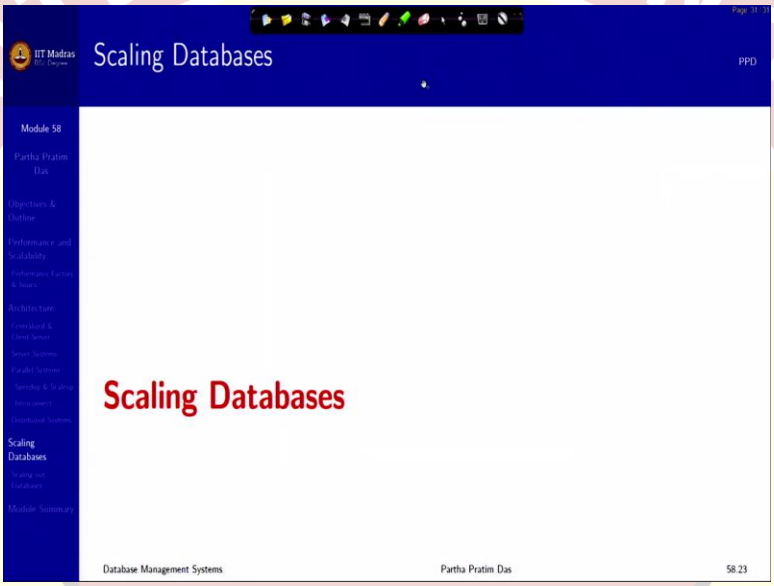
Which is, again can be done in a homogeneous manner. That is, homogeneous means that everywhere at every node, use the same software schema at all sides, same kinds of machines and so on, and just partition the data among the sites or it could be heterogeneous. So, you have different software schema at different sites and different goals. You could process transactions locally at one node or you can process transactions globally across different nodes. So, this is, these are the characteristics of the distributed systems.

(Refer Slide Time: 26:53)



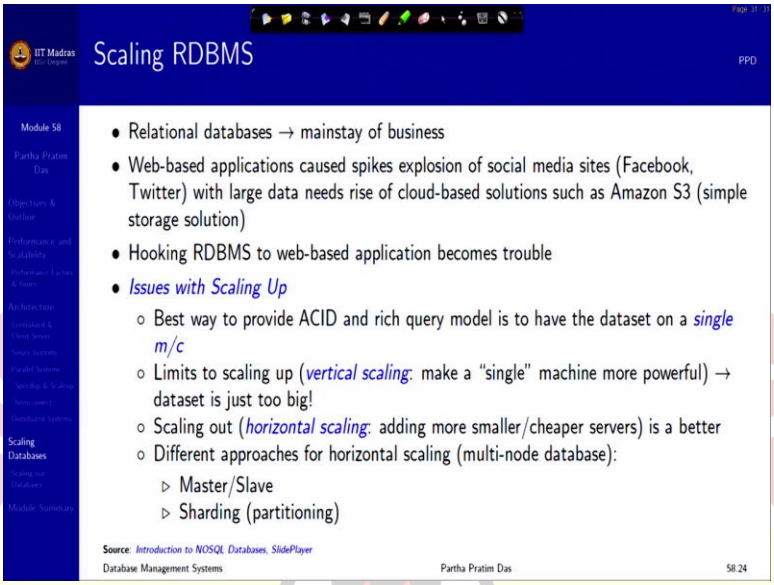
So, the advantages in terms of sharing data, the user at one side is able to access data residing another side So, there is an autonomy of sorts, and it has higher system availability, because of the redundancy because, you may not actually, some data may be replicated across remote sites, some data may be exclusive and so on. But certainly, you have number of disadvantages in terms of development costs, greater potential for bugs. Because the system is getting more complex and increased processing overhead.

(Refer Slide Time: 27:28)



So, given that, let us look at what are the traditional ways or recent ways of scaling up databases.

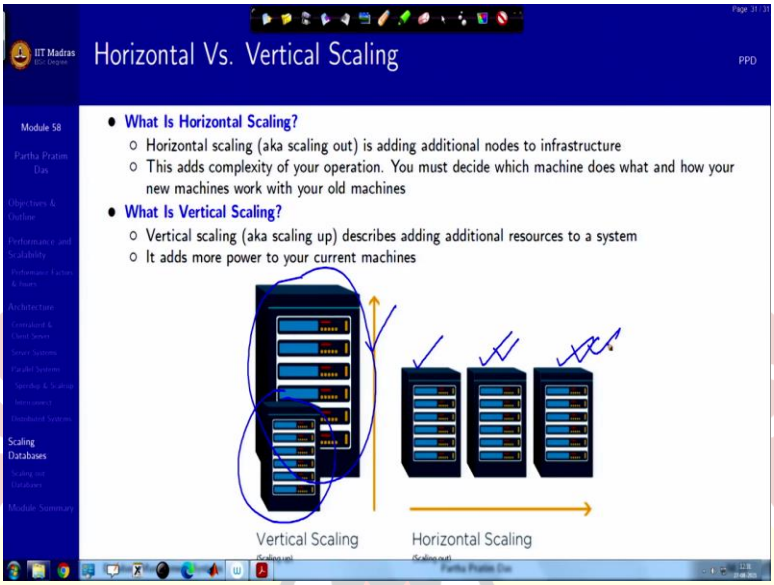
(Refer Slide Time: 27:34)



Relational databases has been the mainstay for business. They have been good for that, but, as we are getting into web 2.0, web-based applications, we are having explosions of social media sites, social media data and so on. Which are much much larger. So, there are several issues with the scaling up. So, it is the best way to provide the ACID is as a rich query model on a single machine, but that does not scale up.

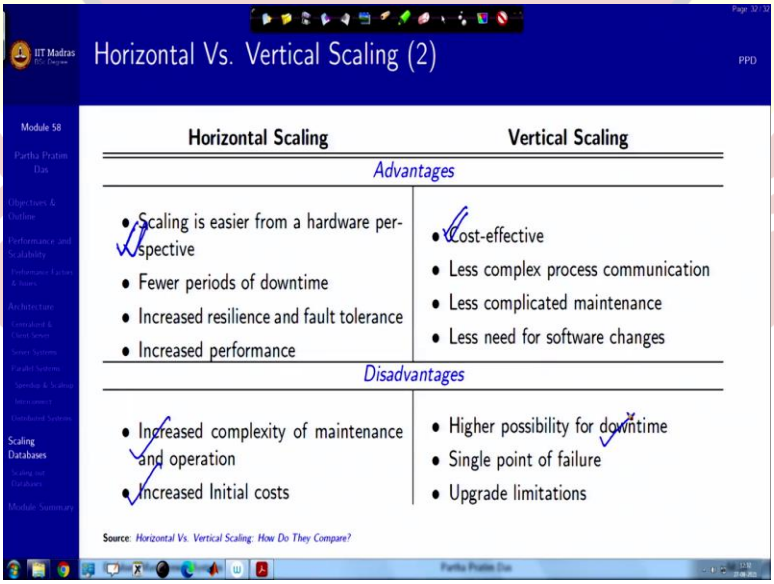
So, you can scale up either vertically by making your machine more and more powerful, have some kind of a supercomputer or mainframe system or you can scale out by expanding it horizontally. And there are different approaches available for that.

(Refer Slide Time: 28:25)



So, this is horizontal versus vertical. So, as you can see, here, you are scaling vertically that is this was your system. And when you need to scale you make it a bigger system, you add more processes, you add more memory faster and all. Or you could scale horizontally in stages, you had one, then you have two then you have three and so on. So, this is since it goes up, this is called scaling up, since this goes horizontally across this is called scaling out.

(Refer Slide Time: 28:58)

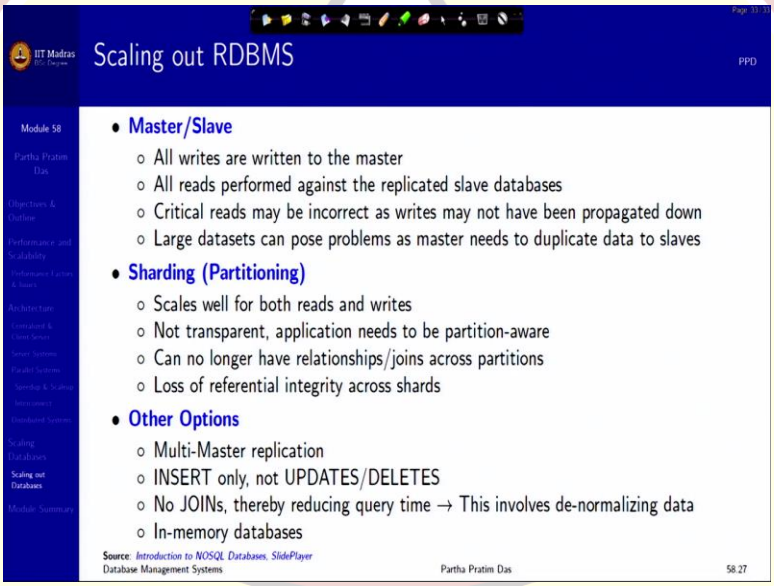


The advantages and disadvantages are very easy to see, naturally vertical scaling is cost effective. Because you have less, you have an existing system. And you are just adding more on

to that. And you can have less complex process communication, less complicated maintenance and so on. There on this side, whereas on the horizontal scaling, you will have increased complexity of maintenance, increased initial cost and so on.

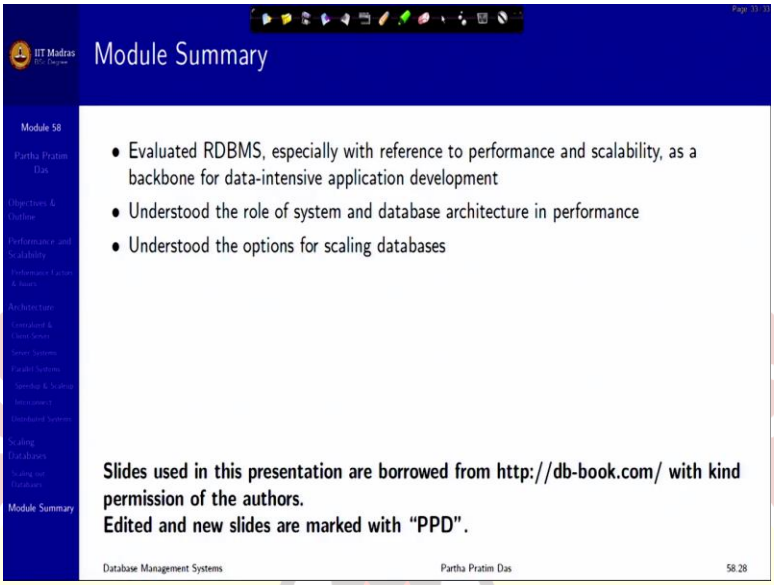
Looking at the advantages of the horizontal scaling, you have naturally scaling is easier because you are adding an independent system. So, your current system is not impacted. These fewer periods of downtime because you have inherent parallelism in the system. It is increased resilience, fault tolerance, increase performance, but naturally vertical scaling has, in contrast, higher possibility of downtime. Because it is a single point of failure.

(Refer Slide Time: 29:57)



So, these are the different, options, particularly scaling out you is turning out to be more popular, where you have master slave system. Where there is one master and multiple slaves. So, which replicate data or there is shredding, which is more popular that which scales well is basically you distribute your data across different processers. And there are mixed options as well.

(Refer Slide Time: 30:20)



So, with that, we come to the conclusion of this module. So, we have tried to take a look at the RDBMS, especially with reference to performance and scalability for different database applications. And assess, tried to assess the alternatives available in terms of architecture, starting from centralized client server to parallel to distributed. And what are the options available for scaling databases. Thank you very much for your attention. We will continue in the next module.